

JAGAIN — Ultimate Product Requirements Document (PRD)

Document status: Product Baseline / V1

Product: JAGAIN

Core promise: Jangan cuma punya, JAGAIN.

Product loop: Catat → Ingat → Rawat

Brand essence: Jaga yang Berarti

0. Executive Summary

JAGAIN is a private, local-first, personal/family asset management app.

- It helps users record important assets, understand condition, remember responsibilities, and act on time.
- It is not merely an inventory app.
- Core problem: people have important things they must remember and maintain, but everything depends on memory.
- Information is scattered across head, WhatsApp, notes, spreadsheets, calendars, receipts, physical docs, or nowhere.
- JAGAIN turns this into: Catat aset → pahami kondisi → catat kejadian → tentukan tanggung jawab → ingatkan saat waktunya → lakukan tindakan → simpan histori.
- Brand identity emphasizes individual/family, not enterprise/business inventory.

1. Product Definition

1.1 Product Name: JAGAIN

Brand meaning: Jaga yang Berarti

1.2 Primary Tagline: Jangan cuma punya, JAGAIN.

1.3 Functional Tagline: Catat – Ingat – Rawat

Phrase	Purpose
"Jangan cuma punya, JAGAIN."	Brand / emotional positioning
"Catat – Ingat – Rawat"	Product explanation / functional positioning

2. Vision

- Become the preferred app in Indonesia for simple, secure, reliable personal/family asset management.
- Product interpretation: JAGAIN as a memory system for important assets, not just a database.

3. Mission

JAGAIN helps users:

1. Record what they own.
2. Remember what needs to be done.
3. Care for assets so they remain valuable and functional.
4. Store information privately and securely.
5. Reduce mental load in asset management.

4. Strategic Product Thesis

- JAGAIN should not be an app for recording assets; it should be an app that prevents users from forgetting to care for them.
- Tracking = mechanism
- Awareness = intermediate outcome
- Action = actual outcome
- Better asset care = ultimate outcome

Three core questions:

- Apa yang saya punya?
- Bagaimana kondisinya?
- Apa yang perlu saya lakukan?

5. Problem Statement

5.1 User Problem

- A. Information fragmentation: WhatsApp, Notes, Spreadsheet, Calendar, documents, receipts, memory; no single source of truth.
- B. Memory burden: service, tax, renewal, warranty, inspections, component replacement.
- C. Lack of asset context: calendar knows “Servis tanggal 15”; JAGAIN knows “Servis mobil Avanza keluarga tanggal 15.”
- D. Lack of history: users often don’t know last service, repair, who did it, recurrence, last condition.

6. Target Audience

Primary Persona: “Family Asset Owner”

- 28–50, family, multiple assets, vehicle, property, electronics, many docs/responsibilities, wants organized life, doesn’t want sensitive data stored carelessly.
- Mental state: “Saya sebenarnya tahu ini penting, tapi terlalu banyak yang harus saya ingat.”

7. Secondary Personas

7.1 Young Professional

- motor, laptop, smartphone, camera, furniture, electronics; needs warranty, maintenance, purchase records, reminders.

7.2 Parents / Household Manager

- family vehicle, AC, fridge, washing machine, house, family devices; needs maintenance, responsibilities, household awareness.

7.3 Student / Young Adult

- laptop, motor, camera, smartphone; needs warranty, service history, ownership records.

7.4 Renter

- personal devices/furniture/electronics/vehicle.

8. Users We Are NOT Designing For

Not:

- enterprise asset management
- warehouse management
- inventory management
- fleet management
- ERP
- procurement system
- business operations software

Even if technically possible, V1 must not move there. Brand is individual/family.

9. Jobs To Be Done

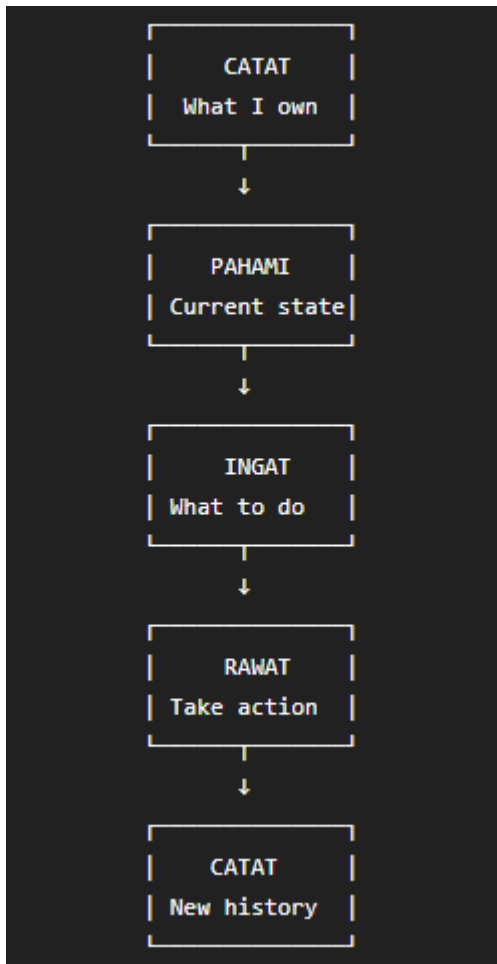
Primary JTBD:

“When I own important things that require ongoing care, I want one simple place to record them and remember what needs attention, so I don't have to rely on memory.”

Functional JTBD examples:

- New vehicle → record it → know condition and obligations.
- Vehicle service due → get relevant reminder → avoid late care.
- Want vehicle history → view activity history.

10. Core Product Loop



Loop should feel natural.

11. Product Principles

1. Reduce Mental Load
Setiap feature harus menjawab:
"Apakah ini membuat hidup pengguna lebih mudah?"
Jika justru menambah administrative work, buang.
2. Simple Beats Complete
Jangan membangun semua kemungkinan.
Concept source secara eksplisit menekankan:
 - simple beats feature-rich
 - kurangi mental load
 - automate memory, not responsibility
 - feature harus membantu user maintain, understand, atau act.
3. Local First
Local-first adalah **product principle**, bukan hanya technical architecture.
Core functionality harus bekerja tanpa internet dan tanpa cloud dependency
4. Privacy Is a Feature
Privacy bukan hanya tulisan di Privacy Policy.

Pengguna harus merasakan:

"Ini data saya."

5. Reminder Must Lead to Action

Reminder yang hanya menghasilkan notification adalah failure.

Reminder ideal:

Reminder → Open Asset → Understand → Act → Complete → History

6. Progressive Enrichment

User tidak boleh dipaksa mengisi 20 field saat pertama kali membuat aset.

Concept menetapkan ideal flow:

pilih tipe → beri nama → save → lengkapi kemudian

12. Product Scope

V1 — JAGAIN Core

Must Have:

1. Asset management
2. Categories
3. Basic asset information
4. Asset condition
5. Asset events/history
6. Reminders
7. Local notifications
8. Search
9. Dashboard
10. Local-first storage
11. Privacy / security controls
12. Backup/export strategy

13. Asset Model

Four layers:

13.1 Static Information

- name, category, brand, model, year, purchase date, identifier, owner.

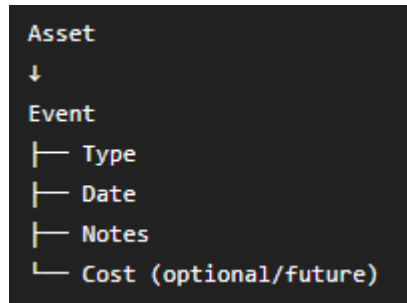
13.2 Current State

- condition, status, location, current user, notes.

13.3 Events/History

- Contoh : service, repair, replacement, inspection, oil change, battery replacement.

- Event:



secara eksplisit membedakan event yang sudah terjadi dari obligation yang perlu dilakukan di masa depan.

13.4 Obligations

- Sesuatu yang harus dilakukan dimasa depan
vehicle tax, insurance renewal, maintenance, registration, warranty expiry, document expiry.

14. Asset Categories

V1:

- Vehicle: Car, Motorcycle, Bicycle, Other vehicle
- Property: House, Apartment, Land, Other property
- Electronics: Smartphone, Laptop, TV, Camera, Appliance, Other electronics
- Personal: Watch, Jewelry, Furniture, Other
- Other: Custom category

Category system must be extensible.

15. Functional Requirements

FR-01 Asset Creation

User dapat membuat asset

- Minimum Required : Category, Asset name
- Optional field : Brand, Model, Year, Purchase date, Owner, Location, Condition, Notes

Acceptance:

- create <30 sec
- not required to fill all details
- asset appears on dashboard after save
- editable anytime

FR-02 Asset Detail

Must answer: What is it? How is condition? What happened? What must be done?

Structure:

Asset Header	
└─	Name
└─	Category
└─	Condition
└─	Status
Information	
└─	Brand
└─	Model
└─	Owner
└─	Location
└─	Purchase date
Upcoming	
└─	Reminders
History	
└─	Events

- Asset Header: Name, Category, Condition, Status
- Information: Brand, Model, Owner, Location, Purchase date
- Upcoming: Reminders
- History: Events

FR-03 Condition

- Good = Kondisi baik
- Needs Attention = Perlu perhatian
- Under Maintenance = Sedang ditangani
- Critical = Kondisi kritis

FR-04 Event / History

User dapat menambahkan event

- Fields: Event type, Date, Notes
- Example: Service / 5 October 2026 / "Servis rutin di bengkel."
- Event harus muncul secara Chronological.

FR-05 Reminder

Merupakan core feature, bukan add-on

- Fields: title, asset, due date, recurrence, notification, completion status

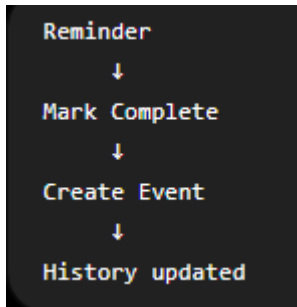
FR-06 Reminder Types

- One-time, contoh : Ganti baterai AC remote — 20 Oct
- Recurring, contoh : Servis motor — setiap 6 bulan
- Annual, contoh : Pajak kendaraan — setiap tahun

FR-07 Reminder Completion

- User can Done or Done + Add to History.

- Ideal:



Reminder → Mark Complete → Create Event → History updated.

Dengan demikian JAGAIN tidak hanya mengingatkan, tetapi membangun histori secara otomatis.

FR-08 Reminder Intelligence

- V1 simple, no AI.
- Statuses: Upcoming, Due Today, Overdue, Completed.
- Capabilities: due date, overdue, upcoming, completed, recurring.

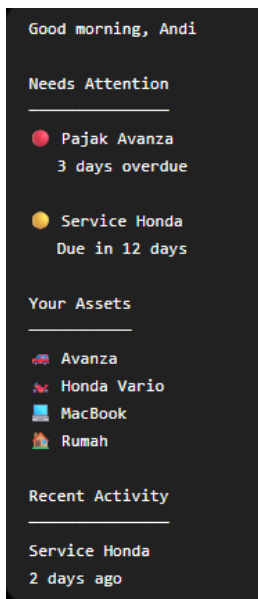
FR-09 Dashboard

Not inventory list; attention-oriented.

Priority:

13. Needs Attention
14. Upcoming
15. Assets
16. Recent Activity

Example:



Good morning, Andi

Needs Attention:

- Pajak Avanza — 3 days overdue

- Service Honda — Due in 12 days

Your Assets:

- Avanza
- Honda Vario
- MacBook
- Rumah

Recent Activity:

- Service Honda — 2 days ago

Critical principle: dashboard tells users what to do, not merely what they own.

16. Search

- Search asset name, category, brand, model, notes, event notes.
- Future: documents, tags, family member.
- V1 local and fast.

17. Asset List

- Search, category filter, condition filter, sort.
- Sort: recently added, recently updated, attention, alphabetical.

18. Onboarding

Onboarding harus sangat pendek.

Screen 1: “Jangan cuma punya, JAGAIN.”

Sub: “Catat aset pentingmu. Ingat apa yang harus dilakukan. Rawat dengan lebih tenang.”

Screen 2 privacy: “Data kamu, di perangkat kamu.”

Screen 3: Create first asset, CTA “Tambah aset pertama”.

No account/email/password/social login/lengthy questionnaire.

Ini konsisten dengan local-first/non-account philosophy pada concept.

19. Empty State Strategy

Not “Belum ada data.”

Example:

“Belum ada yang dijagain.”

“Tambahkan kendaraan, rumah, elektronik, atau aset penting lainnya.”

CTA “Tambah Aset”

20. Notification UX

Actionable.

- Bad: JAGAIN Reminder
- Good: Pajak Avanza jatuh tempo 3 hari lagi
- Action: Lihat Aset
- Control notification fatigue.

21. Data Model

Asset:

Asset	
├─	id
├─	name
├─	category_id
├─	brand
├─	model
├─	year
├─	purchase_date
├─	owner_id
├─	location
├─	condition
├─	status
├─	notes
├─	created_at
└─	updated_at

id, name, category_id, brand, model, year, purchase_date, owner_id, location, condition, status, notes, created_at, updated_at

Category:

Category	
├─	id
├─	name
├─	icon
└─	created_at

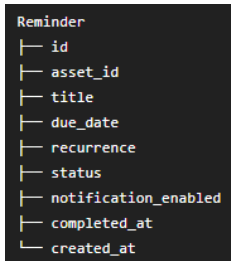
id, name, icon, created_at

Event:

Event	
├─	id
├─	asset_id
├─	type
├─	date
├─	notes
├─	created_at
└─	updated_at

id, asset_id, type, date, notes, created_at, updated_at

Reminder:

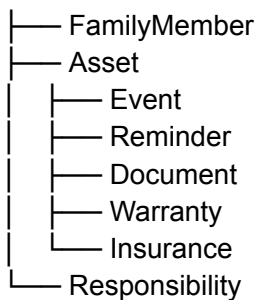


id, asset_id, title, due_date, recurrence, status, notification_enabled, completed_at, created_at

22. Future Data Model

Architecture should allow:

User



Family member and assignment potentially V3.

23. Privacy Architecture

Core: No cloud dependency.

Device → Local database

Not Device → Server → Database

Core app works without internet.

24. Account Strategy

V1 no mandatory account.

- Open app directly.
- Avoid friction, privacy concerns, auth/infrastructure complexity.

25. Backup Strategy

Local-first without backup = risk of data loss.

- V1: local data + manual encrypted export/backup.
- Future: optional encrypted cloud backup, device migration, encrypted file import/export.
- Cloud backup not core dependency.

26. Security

Minimum:

- local database
- secure storage for secrets
- optional app lock
- biometric unlock
- encrypted backup
- no unnecessary telemetry containing asset content
- Sensitive fields not sent to analytics.

27. Local-First Architecture

Technical baseline:

- Mobile: Flutter
- Architecture: Clean Architecture
- State Management: Riverpod
- Database: SQLite / Drift
- Search: SQLite FTS

28. Architecture

Presentation

↓

Application / Use Cases

↓

Domain

↓

Repository

↓

Local Database

↓

SQLite / Drift

Notification:

Reminder → Reminder Scheduler → Local Notification

No V1 cloud API/push notification.

29. Offline Requirement

All core actions work offline:

Action	Offline
Create asset	✓
Edit asset	✓

View asset	✓
Add history	✓
Create reminder	✓
Complete reminder	✓
Search	✓
Dashboard	✓
Notification scheduling	✓

30. Brand & Product Experience

Personality: Trusted, Private & Secure, Meaningful Ownership, Practical, Supportive, Friendly, Modern, Grounded.

UX translation:

- Trusted: no gimmicks
- Private: privacy visible/easy
- Practical: no “cool” features without value
- Supportive: non-judgmental copy
- Friendly: natural Indonesian
- Modern: clean/contemporary
- Grounded: no futuristic/enterprise visuals

31. Visual Direction

Use brand guideline as single source of truth.

- Latest logo; don't redraw/stretch/modify; clearspace and variants.
- Logo philosophy: shield, J, human figure.

32. Typography

- Poppins for headings/brand/prominent UI.
- Inter for body/data/UI details.

33. Color System

- Primary #14B8A6
- Dark #111827
- Secondary #3B82F6
- Light Blue #E0F2FE
- Mint #ECFDF5
- White #FFFFFF

- Grey scale from guideline.
- Brand gradient #14B8A6 → #0F766E.

34. Color Usage Ratio

60% Neutral, 25% Primary, 10% Secondary accent, 5% Supporting accent.

Gradient only hero/highlight/premium CTA/accent, not whole-screen backgrounds.

Jangan menjadikan gradient sebagai warna utama seluruh aplikasi.

Gradient adalah:

- hero
- highlight
- premium CTA
- accent

bukan background semua screen.

35. Visual Anti-Patterns

Must not look like warehouse software, logistics software, fleet management, ERP, enterprise dashboard, inventory system.

- Avoid truck, warehouse, gudang, forklift, factory, corporate logistics imagery.

JAGAIN = people + meaningful ownership + care.

36. UX Information Architecture

Navigation:

Home

Assets

Reminders

Activity

Settings

Home = action-oriented dashboard

Assets = asset database

Reminders = action center

Activity = history

Settings = privacy/backup/preferences

37. Core Navigation

Same structure: Home / Assets / Reminders / Activity / Settings.

38. Critical User Journeys

A First Asset

Open App → Welcome → Add Asset → Choose Category → Asset Name → Save → Asset Detail

Target <30 seconds.

B Add Reminder

Asset → Add Reminder → Title → Date → Repeat → Notification → Save

C Complete Reminder

Notification → Open → Asset → Reminder → Complete → Add to History → Done

D Review Asset

Home → Asset → Condition → Upcoming → History

39. MVP Feature Matrix

Feature	Priority	V1
Create asset	P0	✓
Edit asset	P0	✓
Delete asset	P0	✓
Categories	P0	✓
Asset condition	P0	✓
Asset detail	P0	✓
Event history	P0	✓
Reminder	P0	✓
Recurring reminder	P0	✓
Local notification	P0	✓
Dashboard	P0	✓
Search	P1	✓
Filter	P1	✓
App lock	P1	✓
Backup/export	P1	✓

Family members	P2	✗
Shared household	P2	✗
Documents	P2	✗
Warranty	P2	✗
Insurance	P2	✗
AI	P3	✗
Cloud sync	P3	✗
Web app	P3	✗

40. What We Explicitly Do NOT Build

V1 excludes:

- AI chatbot
- AI assistant
- AI asset recognition
- social feed
- social sharing
- gamification
- achievement
- leaderboard
- public profiles
- cloud dependency
- mandatory login
- marketplace
- ecommerce
- accounting
- inventory
- warehouse
- fleet management
- advanced analytics
- enterprise dashboard

41. Success Metrics

North Star Metric: Reminder Completion Rate

Formula: Completed reminders / Due reminders

Example: 100 due, 72 completed = 72%.

Reason: value is helping users remember and complete responsibilities, not merely adding assets.

42. Supporting Metrics

Activation:

- first asset
- first reminder

Activation event = Asset + Reminder created

Engagement:

- WAU
- assets/user
- reminders/user
- reminder completion
- asset detail views
- history entries

Retention:

- D7, D30, D90

Read retention relative to reminder cycles.

43. Product Health Metrics

Good: high reminder completion, low notification dismissal, recurring reminder usage, users returning after reminders.

Bad: many assets but no reminders; many reminders but never completed; many notifications but app not opened; onboarding abandonment; many asset deletions.

44. Key Product Hypotheses

- H1 users have real difficulty remembering asset responsibilities.
- H2 asset-specific reminder more useful than generic calendar reminder.
- H3 local-first/privacy increases trust.
- H4 users will record assets if entry friction is very low.
- H5 reminder completion is leading indicator of product value.

45. Validation Plan

Qualitative 10–15 family asset owners.

Ask what they own, what must be maintained, what they've forgotten, how they remember, spreadsheet/calendar/notes use, what's annoying.

Do not ask "Will you use JAGAIN?" because it produces polite answers, not evidence.

46. Prototype Validation

Test:

17. Create asset — complete without help.
18. Create reminder — understand reminder ↔ asset.
19. Receive reminder — know what to do.

47. Launch Criteria

Product:

- first asset creation smooth
- reminders work reliably
- notifications reliable
- history reliable
- local data persistent
- backup/export functional

UX:

- onboarding completion acceptable
- no major usability blocker
- users understand value prop

Technical:

- no critical data-loss bug
- no notification scheduling bug
- app usable offline
- database migration tested

48. Quality Requirements

Reliability: no data loss on create/edit/delete.

Performance:

- cold start <3 sec on supported mid-range device
- asset list render <300ms after data available
- search <200ms for normal local dataset

Offline: core flows 100% offline.

Accessibility: readable text, sufficient contrast, appropriate touch targets, semantic labels, dynamic font support where practical.

49. Analytics & Privacy

Allowed:

- asset_created
- reminder_created
- reminder_completed
- reminder_overdue
- asset_viewed

Not allowed:

- asset name
- license plate
- notes
- address
- personal document content
- sensitive asset details

V1 can start with minimal analytics and clear consent.

50. Monetization

V1 free.

Don't monetize too early; prove usefulness first.

Tambahkan Page terpisah dibelakang yang intinya ucapan terimakasih sudah menggunakan aplikasi kami, salam hangat dari developer.

Jika anda merasa apps ini membantu anda bisa support developer kami via Gopay : 0851 797 65891

Custom apps available, contact 0851 797 65891, code name smughbowf. Tolong ini dri

51. Potential Future Monetization

Option A Premium:

- encrypted cloud backup
- multi-device sync
- advanced documents
- household sharing
- advanced reminders

Option B One-time customization:

- custom branding
- custom theme
- custom fields
- custom workflows

Custom version is an opportunity but must not become unlimited freelance work.

52. Roadmap

V1 — JAGAIN: “Know what you own.”

- assets
- categories
- details
- condition
- reminders

- basic history

V1.1 — Reliability & Care

- better reminder UX
- richer history
- backup
- app lock
- search improvements
- notification improvements

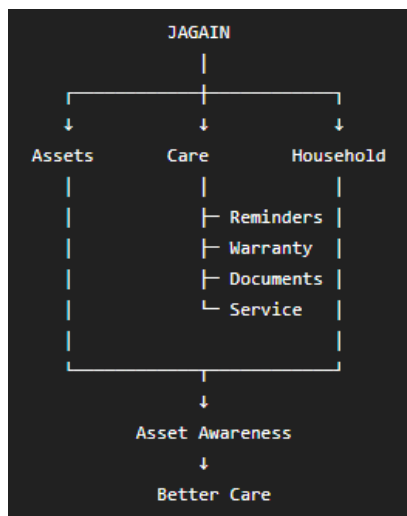
V2 — Asset Care: “Take care of what you own.”

- richer maintenance
- service history
- advanced reminder rules
- documents
- warranty
- insurance

V3 — Household: “Manage what your family owns.”

- family members
- asset assignment
- responsibilities
- shared household awareness

53. Long-Term Product Architecture



Destination, not V1 scope.

54. Biggest Risks

Risk 1 “This is just inventory app.” Critical. Mitigation: attention-first UX.

Risk 2 User lazy to input. Critical. Mitigation: progressive enrichment.

Risk 3 Notification fatigue. High. Mitigation: prioritization + smart defaults.

Risk 4 Local-only data loss. Critical. Mitigation: manual encrypted backup early.

Risk 5 Scope explosion. Critical. Mitigation: strict V1 boundary.

55. The Most Important Product Decision

Do not measure success by number of assets recorded.

100 assets + no actions = failure.

10 assets + 15 reminders + 13 completed = more valuable.

Primary metric = Reminder Completion Rate.

Activation = First Asset + First Reminder.

Product outcome = User takes action on something they might otherwise forget.

56. Product North Star

“JAGAIN membantu pengguna mengetahui apa yang mereka punya, memahami kondisinya, dan tidak lupa melakukan apa yang perlu dilakukan.”

If a feature does not help one of these three, don't build it.

57. Final Product Definition

JAGAIN is a private, simple, local-first personal/family asset management app. It helps users record what they own, understand asset condition, remember care obligations, and store action history—so they don't merely own something, but actually take care of it.

Brand: Jaga yang Berarti

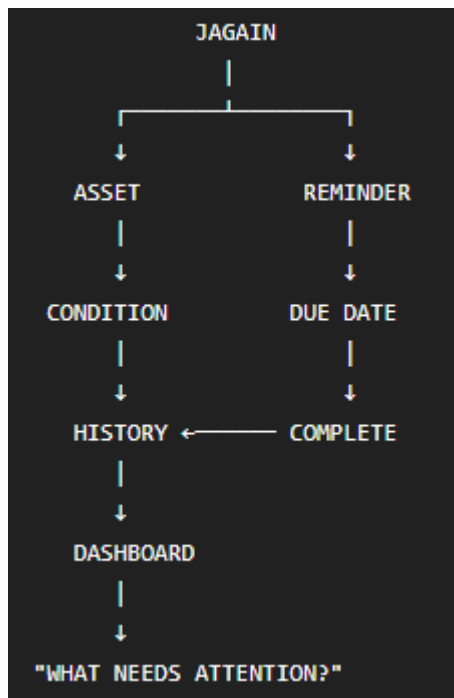
Promise: Jangan cuma punya, JAGAIN.

Product loop: Catat – Ingat – Rawat

North Star: Help users take care of what matters before they forget.

58. Ultimate MVP Definition

If the entire PRD is reduced to the minimum product that proves the thesis:



This is the product to build first.

Not cloud platform, AI, social app, family ERP, or inventory system.

One problem first: helping humans remember to care for things that matter to them.

And that is sharper than “asset tracker.”